Reviewer Pack — Minimal Rank of Braided Group Representations and Resolution…

Entry 864deba2fadd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 06:50:45 UTC

Minimal Rank of Braided Group Representations and Resolution Proof Width

Entry ID: `864deba2fadd` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 06:50:45 UTC

1. Conjecture

Field A (mathematical branch): Braids in Group Theory **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

> The minimal rank of a braided group representation is linearly correlated with the resolution proof width of its corresponding CNF formula, such that the minimum rank r(G) of a braided group G associated with a CNF φ_G satisfies r(G) = Θ(w(φ_G)), where w(φ_G) denotes the resolution proof width of φ_G.

Rationale (proposer's reasoning):

> Braids in group theory have been studied for their applications in quantum computation, particularly as a resource for entanglement. The representation theory of braided groups could provide a novel perspective on the complexity of satisfiability problems by revealing hidden structures that affect proof length.

Taxonomy category: `Braids in Group Theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `be990279dec22880`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for at least 24 out of 30 random seeds, linear regression analysis yields a correlation coefficient ≥ 0.95 and an r(G) value within ±10% of w(φ_G). It is falsified if the correlation coefficient < 0.7 or any seed results in |r(G) - w(φ_G)| > 20%.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `minimal rank braided group representation AND boolean satisfiability resolution proof complexity` - `CNF formula minimal rank braided group representation related to resolution proof width` - `θ(w(φ_G)) associated with braided group G in resolution proof complexity`

Top relevant hits considered: - [http://arxiv.org/abs/2501.16039v4] Complexity of Constructing Minimal Faithful Permutation Representations for Fitting-free Groups - [http://arxiv.org/abs/1405.2700v1] Zero Excess and Minimal Length in Finite Coxeter Groups - [http://arxiv.org/abs/1908.06409v1] Schur multipliers of special p-groups of rank 2 - [http://arxiv.org/abs/hep-ph/0610012v1] Tevatron-for-LHC Report of the QCD Working Group - [http://arxiv.org/abs/0901.0512v4] Expected Performance of the ATLAS Experiment - Detector, Trigger and Physics - [http://arxiv.org/abs/1808.08676v3] Constraining the p-mode–g-mode tidal instability with GW170817 - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \to \mu^+\mu^-$ decay from the combined analysis of CMS and LHCb data

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_braided_group(n):
        # Placeholder for generating a braided group
        return [random.randint(1, n) for _ in range(n)]

    def construct_cnf_formula(group):
        # Placeholder for constructing a CNF formula from a braided group
        cnf = []
        for i in range(len(group)):
            for j in range(i + 1, len(group)):
                if group[i] != group[j]:
                    cnf.append([i + 1, -j - 1])
        return cnf

    def compute_minimal_rank(group):
        # Placeholder for computing the minimal rank of a braided group
        n = len(group)
        rank = 0
        while True:
```

```
            found = False
            for i in range(n):
                if group[i] not in group[:i]:
                    rank += 1
                    break
            else:
                return rank

    def compute_resolution_proof_width(cnf):
        # Placeholder for computing the resolution proof width of a CNF formula
        n = len(cnf)
        width = 0
        while True:
            found = False
            for clause in cnf:
                if len(clause) > width:
                    width = len(clause)
                    break
            else:
                return width

    results = []
    for _ in range(30):
        n = random.randint(5, 40)
        group = generate_braided_group(n)
        cnf = construct_cnf_formula(group)

        rank = compute_minimal_rank(group)
        proof_width = compute_resolution_proof_width(cnf)

        results.append({
            "metric_name": "correlation_coefficient",
            "metric_value": rank / proof_width if proof_width > 0 else None,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": ""
        })

    correlation_coefficient = sum(r["metric_value"] for r in results) / len(results)
    conjecture_holds = correlation_coefficient >= 0.95 and all(abs(r["metric_value"] - proof_width

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(r["n_max"] for r in results),
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "insufficient_instances"
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] + list(rang
    results = []

    for seed in seeds:
```

```
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    elif any(abs(r["metric_value"] - proof_width) > 20 for r in results):
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if abs(r["metric_value"]
        print(f"RESULT: FALSIFIED counterexample='insufficient_instances' first_failing_seed={firs
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_instances")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it was unable to complete the required linear regression analysis. | next: Retry the experiment with a longer timeout or increase system resources to ensure the test can complete.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13726
2	propose	ollama_remote	glm4:latest	0	0	13324
3	preregistration	ollama_remote	glm4:latest	0	0	10005
4	novelty	ollama_remote	glm4:latest	0	0	12007
5	novelty	ollama_remote	glm4:latest	0	0	9417
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17297
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13080
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12783
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10502
10	judge	ollama_remote	glm4:latest	0	0	11560

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 123702 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/864deba2fadd.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/864deba2fadd.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/864deba2fadd.tar.gz` (if generated)